

NAIRR PILOT CLASSROOM EXPANSION CONFERENCE

Research-Emerging Institutions with Large Computing Programs

Scaling AI Education for Engineers

A Project-Based Deep Learning Course and GenAI-Enhanced Teaching Tools at Texas A&M

Yang Liu

Assistant Professor, Department of Nuclear Engineering

SMART Lab · GAISE Lab · Texas A&M University

SCIENTIFIC MACHINE LEARNING FOR ADVANCED REACTOR TECHNOLOGIES (SMART LAB)



Generative AI for Science and Engineering (GAISE) Lab, Texas A&M Institute of Data Science



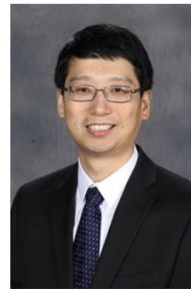
Dr. Yang Liu, Director
Department of Nuclear
Engineering



Dr. Shuiwang Ji, Co-Director
Department of Computer
Science and Engineering



Dr. Raymundo Arroyave
Department of Materials
Science and Engineering



Dr. Xiaofeng Qian
Department of Materials
Science and Engineering

Generative AI
Integrate GenAI
with domain
knowledge for
science and
engineering
problems



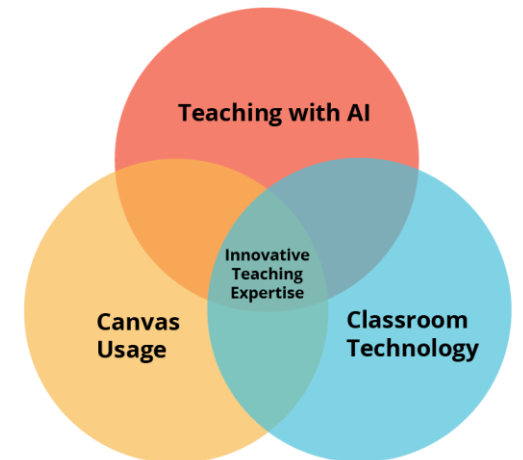
**Physics-informed
Machine Learning**
Integrate physics-
informed machine
learning models
with nuclear M&S
tools to ensure
accuracy and
efficiency

**Digital Twin Assisted
Experiment**

An integrated
hardware/software
testbed for flexible
tasks

**Texas A&M Center of
Teaching Excellence**

**Innovative Teaching
Fellow – AI Track**



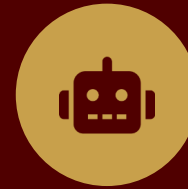
Two parts: the course, and the teaching



PART I

The Course

- Why engineering students need their own DL course
- Course design, enrollment, and three-year evolution
- The core: deep learning for scientific computing
- A project-oriented model that feeds student research
- Outcomes and publishable results



PART II

GenAI-Enhanced Teaching

- Using LLMs to lower the coding barrier for students
- Strengthening course materials with generative AI
- Where we take GenAI-assisted teaching next
- Closing: GenAI is a tool, not the whole of deep learning

A gap in graduate AI education

Strong DL instruction at TAMU



Deep Learning



Reinforcement Learning



Generative AI

All are designed for computer science students, assuming heavy programming and CS theory.

Engineering graduate students are left out

- They increasingly need deep learning for research: modeling, simulation, and data analysis
- Many lack the CS prerequisites and day-to-day coding fluency the existing courses assume
- Vision and language examples feel disconnected from physics-based engineering problems

The gap: no deep learning course built for engineers outside computer science.

A course designed for the way engineers learn



Built on math, not a CS degree

We start from undergraduate calculus and linear algebra and build coding from a Python recap in week one.



Framed by engineering

Every method is taught through modeling and simulation, flow and heat transfer, and complex system analysis.



Anchored in scientific computing

We connect deep learning to the simulation problems engineers already solve, the focus CS courses skip.



Hands-on and research-relevant

Colab notebooks, in-class coding, and a project each student aims at a problem from their own field.

THE COURSE

Deep Learning for Engineering Applications



Launched Spring 2024 Offered three times to date and now a permanent course.



Flexible format In person plus synchronous and asynchronous online delivery.



Accessible prerequisites Undergraduate calculus and linear algebra; Python recommended, taught from a recap.



Free, open tools Python, PyTorch, Jupyter, and Google Colab, all open source.



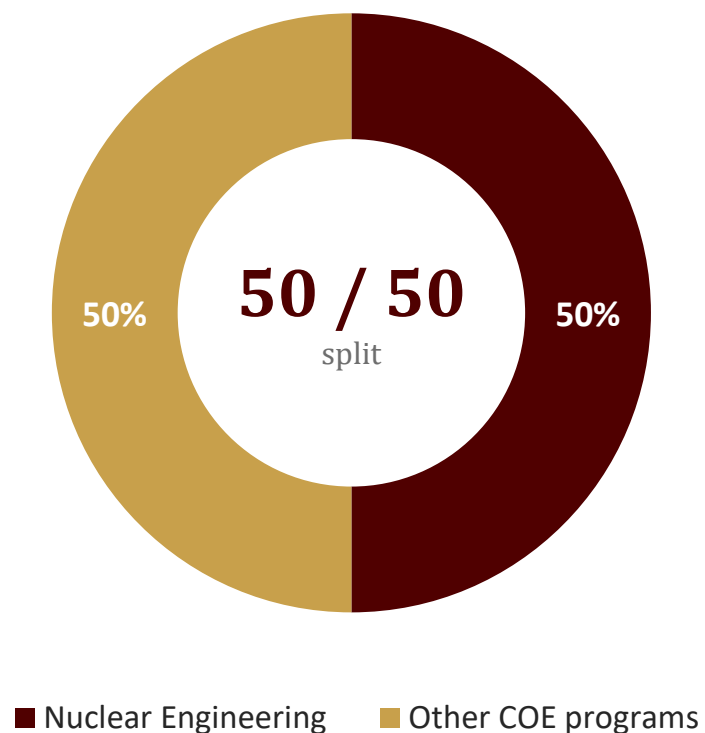
Engineering audience Graduate students across the College of Engineering, with an undergraduate track added later.

“

**A deep learning
course built for
engineers, not for
computer scientists.**

**NUEN 689 → NUEN 489/689 → NUEN
467/667**

Who takes the course



≈ 30

students per year, stable across offerings



3

times offered since Spring 2024



2

levels reached: graduate and undergraduate

Three offerings, refined from student feedback

2024

NUEN 689

GRADUATE SPECIAL TOPICS

- Three-part structure: fundamentals, embedding physics, advanced topics
- Project-weighted to reward research-style work

Homework 40% • Project 60%

2025

NUEN 489/689

UNDERGRADUATES ADDED

- Added a Part 0 onboarding: Python and Colab tutorial, probability basics
- Expanded generative AI and large language model coverage

Homework 60% • Project 40%

2026

NUEN 467/667

PERMANENT COURSE NUMBER

- Four in-class coding sessions for hands-on practice
- Physics block renamed Deep Learning for Scientific Computing; added nuclear topic and LLM agents / RAG

Homework 40% • Coding 15% • Project 45%

Each change responded to feedback: more onboarding for non-CS students, more hands-on coding, a clearer scientific-computing identity.

Most Recent Syllabus

Course schedule		
Week	Topic	
	Weekly lecture 1	Weekly lecture 2
Part 0: Knowledge recap		
1	Course introduction	Recap on linear algebra and probability basics
Part 1: Deep learning: theory and models		
2	Linear regression and logistic regression	Regularization and optimization (HW 1)
3	Backpropagation	Training neural networks -I
4	Training neural networks -II	Recurrent neural networks (HW 1 Due, Project idea discussion)
5	Convolutional neural networks	Deep learning framework: PyTorch (HW 2)
6	Coding session: train a model with PyTorch	Characterization of machine learning models (Project proposal)
Part 2: Deep learning for scientific computing		
7	Physics-informed neural networks – I	Physics-informed neural networks – II (HW 2 Due)
8	Neural operators – I	Neural operators – II
9	Differentiable hybrid modeling	Coding session: PINNs and neural operators (HW 3)
Course schedule		
10	Neural ODE	Uncertainty quantification for deep neural networks
11	Gaussian processes and Bayesian optimization for <u>AutoML</u>	Special topic: Deep learning for Nuclear Engineering Applications (HW 3 Due) (HW 4) (Milestone report)
Part 3: Introduction to generative AI		
12	Introduction to attention mechanism, Transformer, and large language model	Generative AI for science and engineering
13	Coding session: LLM agents and RAG	Project presentation - I (HW 4 due)
14	Project presentation – II	Project presentation – III



PART I · THE DIFFERENTIATOR

Deep Learning for Scientific Computing

The module a typical computer science deep learning course does not cover.

Where engineering deep learning diverges from CS

CS deep learning focuses on

- Vision, language, recommendation
- Generative media at internet scale
- Abundant labeled data, soft accuracy targets

Engineering problems are governed by

- Conservation laws, PDEs, dynamical systems
- Expensive simulations and sparse, noisy data
- Hard requirements on reliability and safety

Our scientific-computing core teaches methods built for this setting:

1
Physics-informed
neural networks

2
Neural
operators

3
Differentiable
hybrid modeling

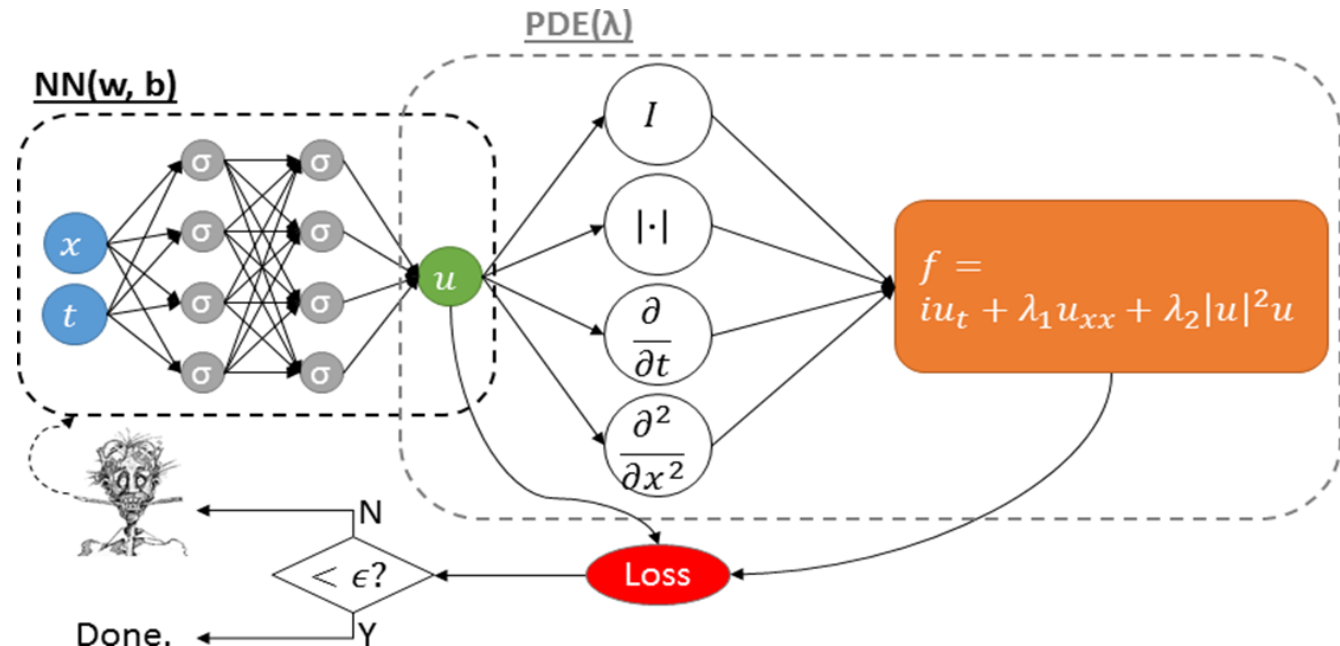
4
Neural
ODEs

5
Uncertainty &
Bayesian methods

Physics-Informed Neural Networks (PINNs)

Encode the equations into the loss. Governing PDEs enter training directly through automatic differentiation.

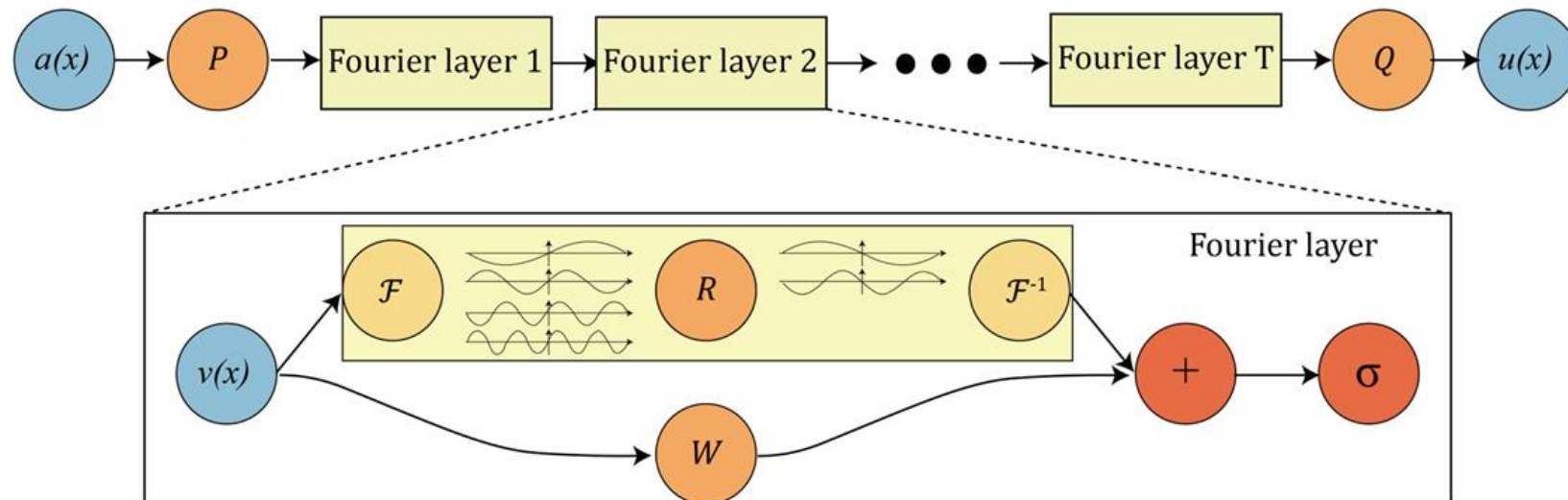
- Solve forward and inverse problems with little or no labeled data
- Engineering uses: heat conduction, fluid flow, structural mechanics, and parameter identification from sparse sensors
- In class: students implement a PINN for a canonical PDE, then extend it to a problem from their own field



Neural Operators

Learn maps between functions. Operators act on whole input fields, not single points.

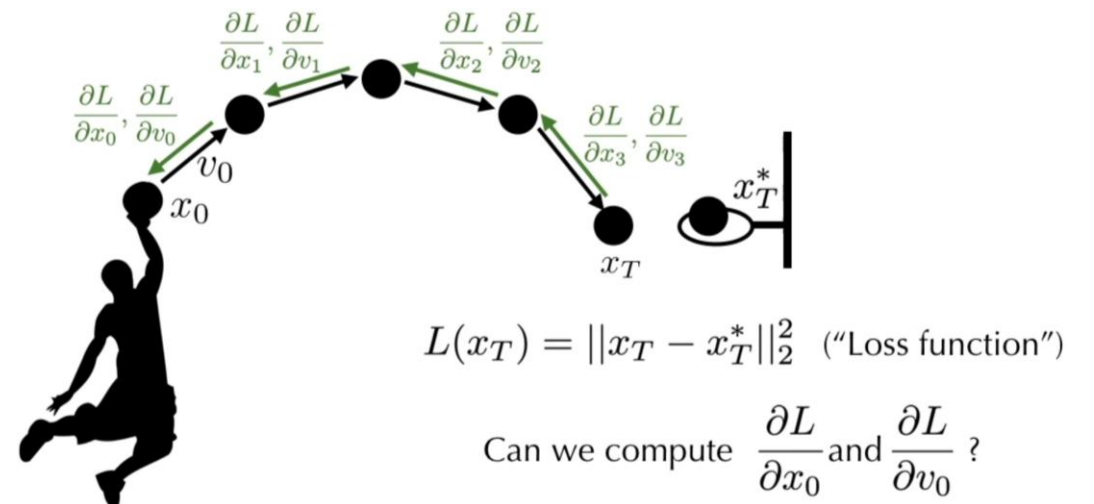
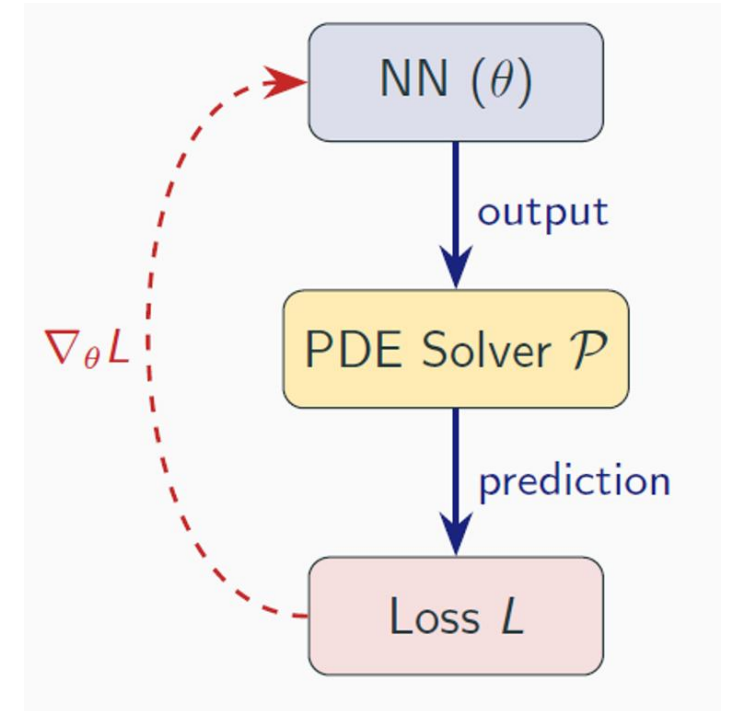
- One trained model generalizes across boundary conditions, source terms, and geometries
- Examples: DeepONet and the Fourier Neural Operator
- Engineering uses: fast surrogates that replace repeated, expensive PDE solves in design and uncertainty studies
- In class: students train an operator surrogate and compare cost and accuracy against a numerical solver



Differentiable Hybrid Modeling

Combine physics with learning. Known equations and learnable terms live in one differentiable program.

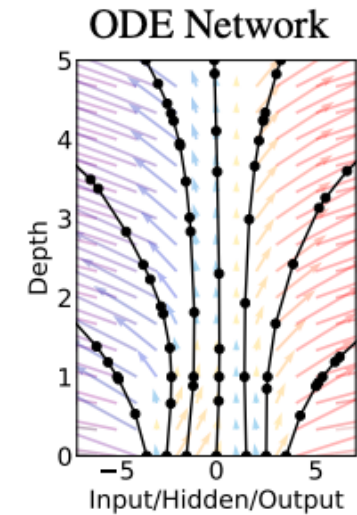
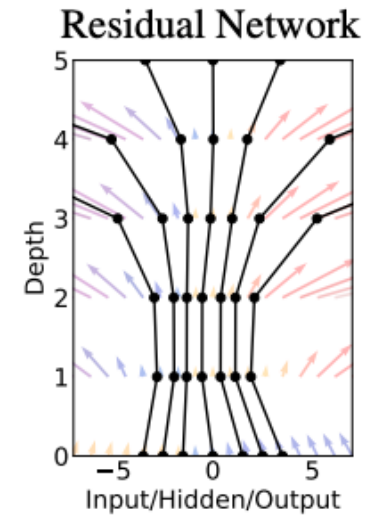
- Gradients flow through the solver, so physics and data train together
- Engineering uses: closure models, reduced-order models, system identification, and digital twins
- In class: students build a hybrid model that corrects a simplified physics model using data



Neural ODE

Treat depth as continuous time. Hidden states follow a learned vector field that an ODE solver integrates, in place of a fixed stack of layers.

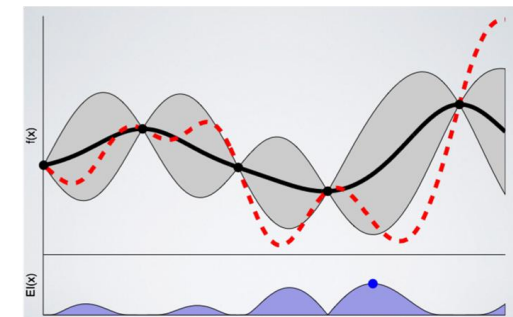
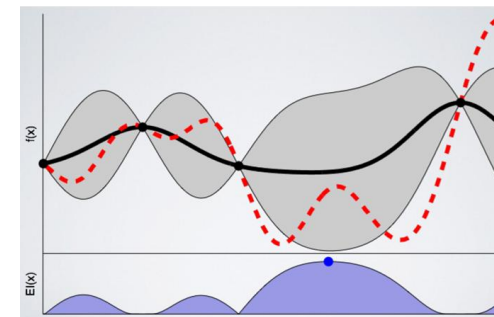
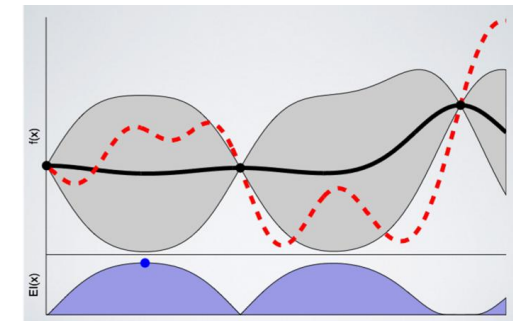
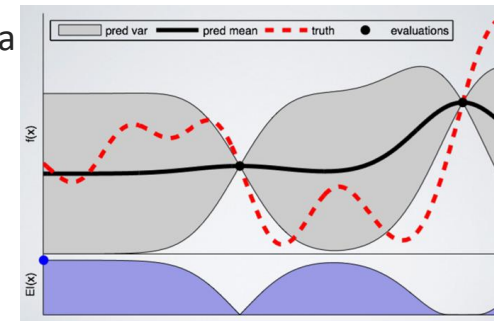
- A residual network is the discrete case; a neural ODE learns the continuous dynamics between layers
- Engineering uses: continuous-time dynamics, irregularly sampled sensor data, and memory-efficient training through the adjoint method
- In class: students train a neural ODE on a dynamical system and compare it against a discrete-step baseline



Bayesian Optimization and Uncertainty Quantification

Quantify uncertainty, then act on it. A probabilistic surrogate predicts with confidence bands, and an acquisition function turns those bands into the next evaluation.

- Gaussian-process surrogates return a calibrated mean and variance, not a single point prediction
- Engineering uses: model and hyperparameter search (AutoML), experimental design, and decisions under sparse, expensive data
- In class: students quantify uncertainty on their own model, then tune it with Bayesian optimization



Project-oriented by design

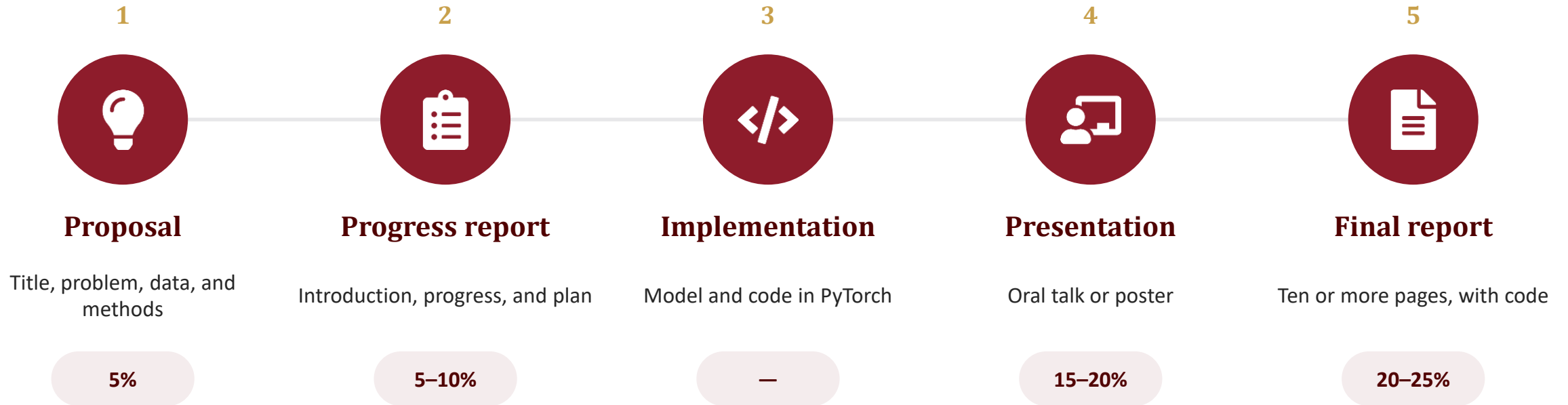
- One substantial project anchors the course
- Students define their own project from their research or domain, rather than a fixed assignment
- When a student has no project, we assign a paper to reproduce
- Flexible teams: work solo or in pairs; undergraduates can join a graduate project
- The goal: every student applies deep learning to a problem they actually care about

“

**Feedback is consistent:
the project directly
advances the student's
own research.**

The course doubles as a research accelerator.

From idea to result: the project arc



Milestone weights vary by offering; the arc stays the same.

Projects connect to real research

Many projects mature into thesis work. Representative project themes across the engineering cohort:



Thermal-fluid surrogates

Physics-informed and neural-operator models for flow and heat transfer, replacing expensive CFD runs.



System monitoring

Graph and neural-ODE models for digital-twin-style state prediction of engineering systems.



Simulation automation

LLM-agent workflows that set up, run, and analyze engineering simulations.

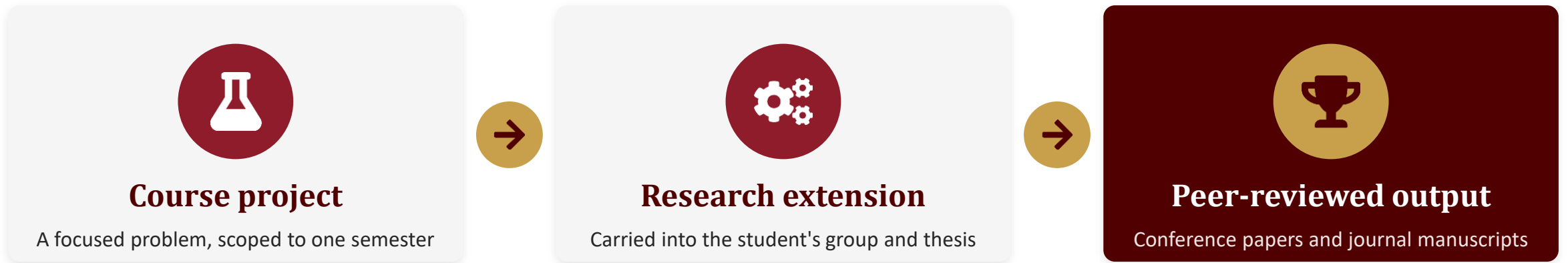


Inverse design

Generative and diffusion models for design and reconstruction under physical constraints.

Representative examples; replace with specific student projects when presenting.

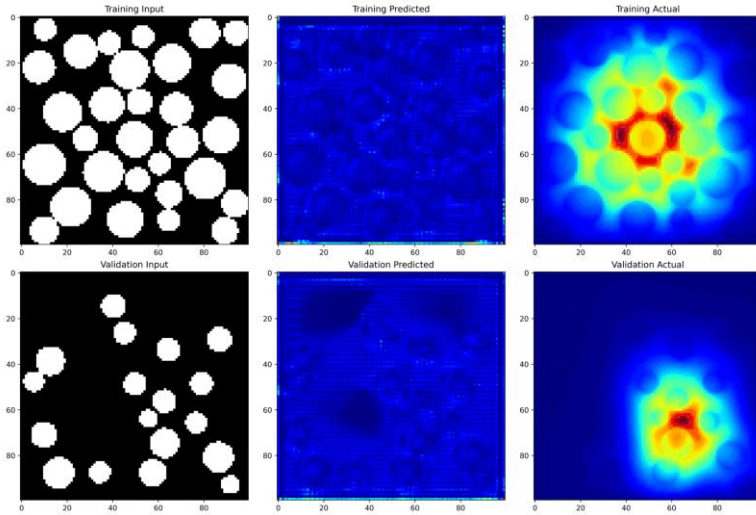
From course project to publication



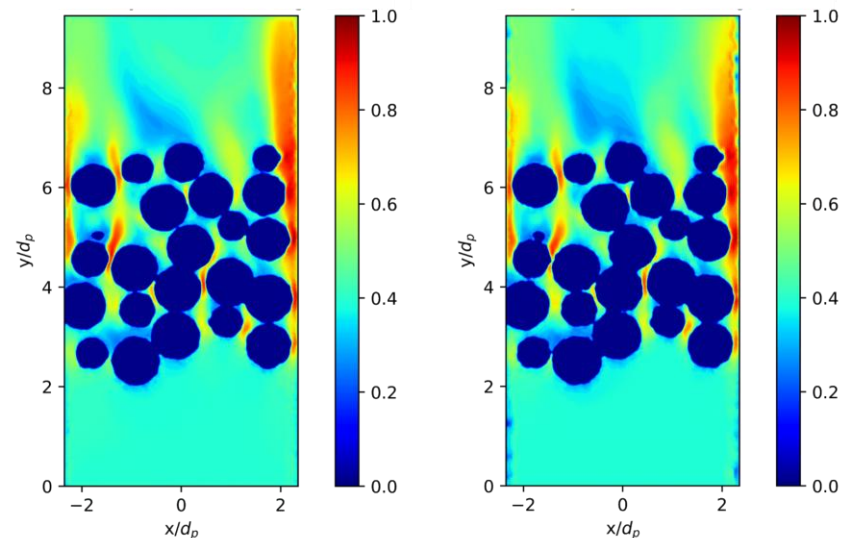
Where engineering DL course work tends to land

Conferences: ANS Annual and Winter Meetings, domain workshops
Journals: Progress in Nuclear Energy, Nuclear Engineering and Technology, Knowledge-Based Systems, and similar venues

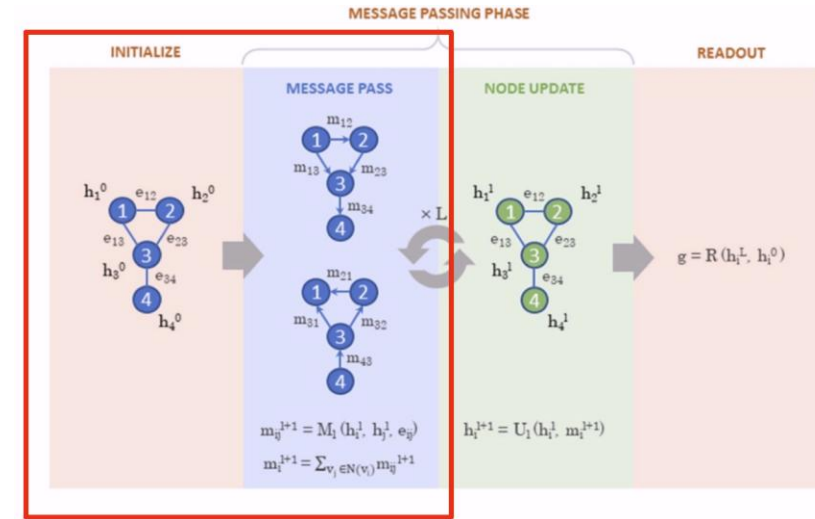
Course projects demonstration



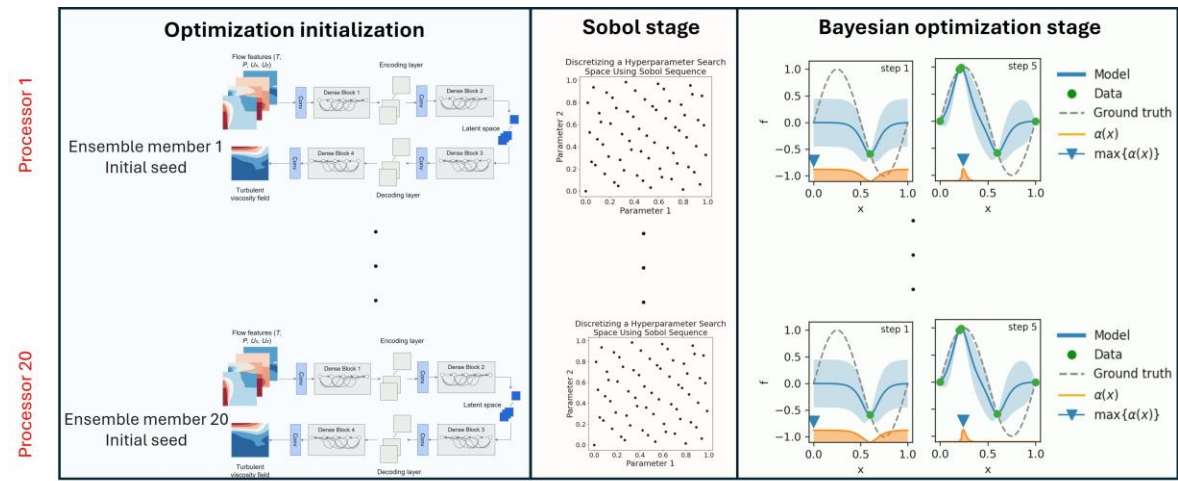
Neutron flux prediction with CNN



Pebble bed flow prediction with DCNN

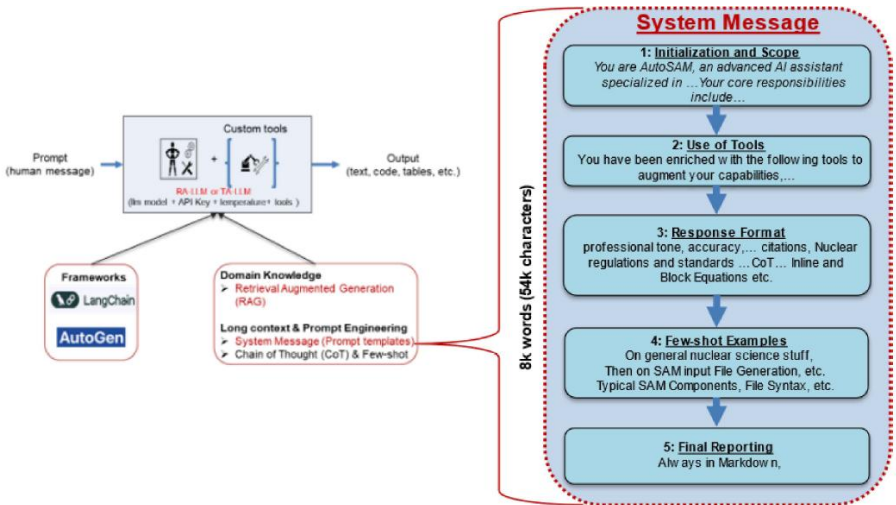
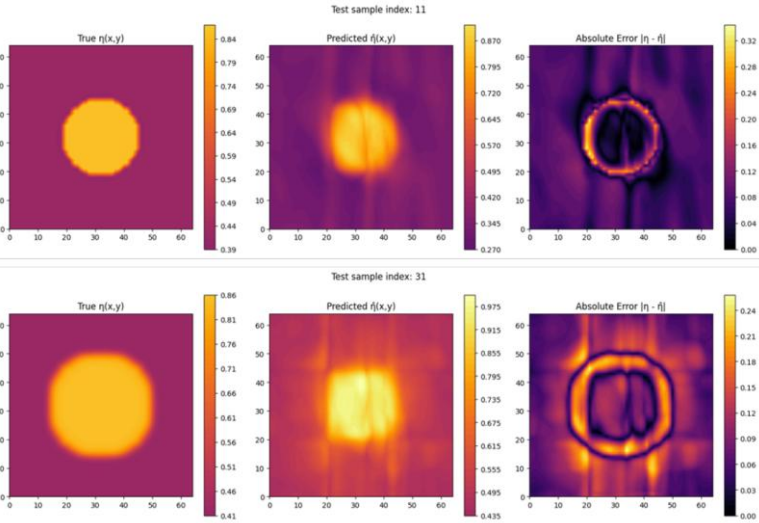


Material interatomic potential prediction with GNN

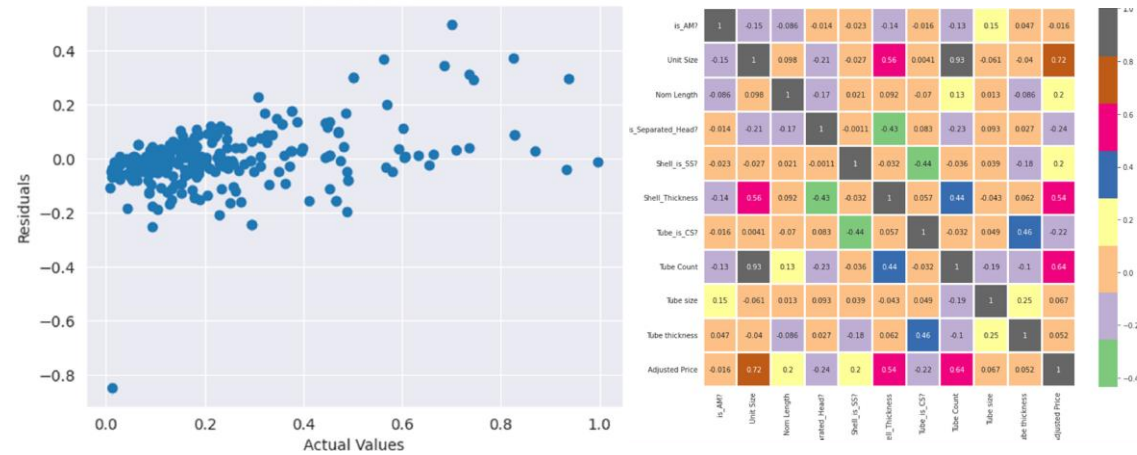


Bayesian optimization for SFR flow and heat transfer

Course projects demonstration (Cont'd)

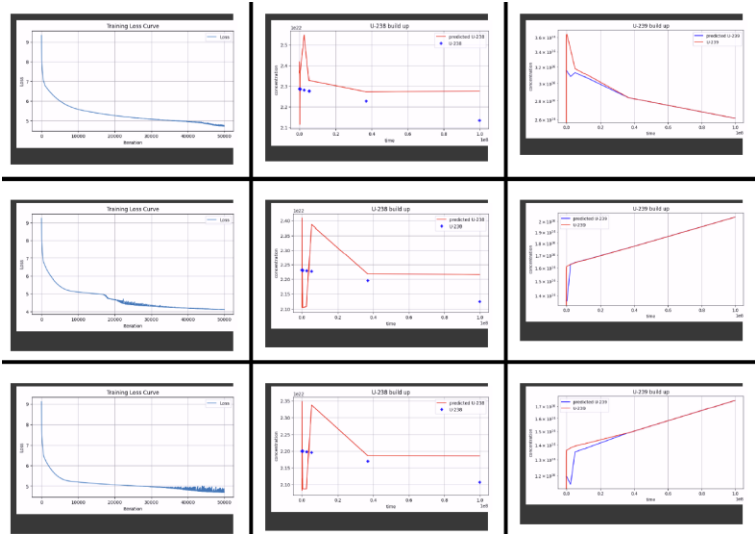


Materials phase transformation prediction using neural operator



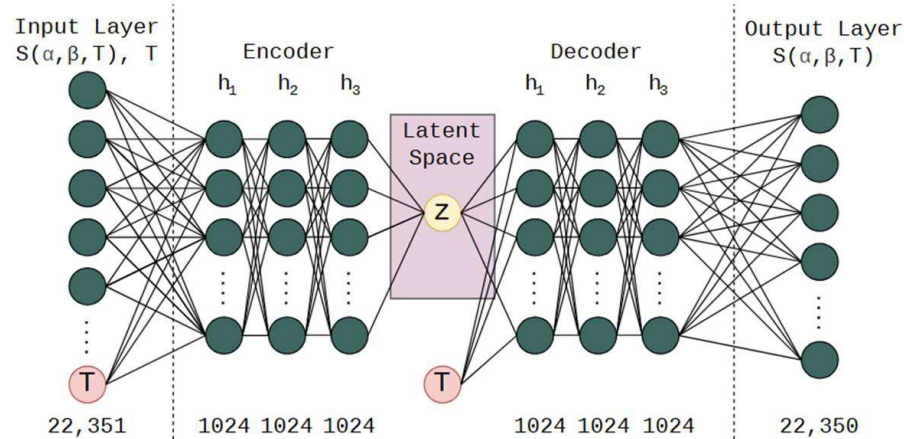
Cost estimation for fabricating specialized heat exchangers

Automating SAM workflow with LLM

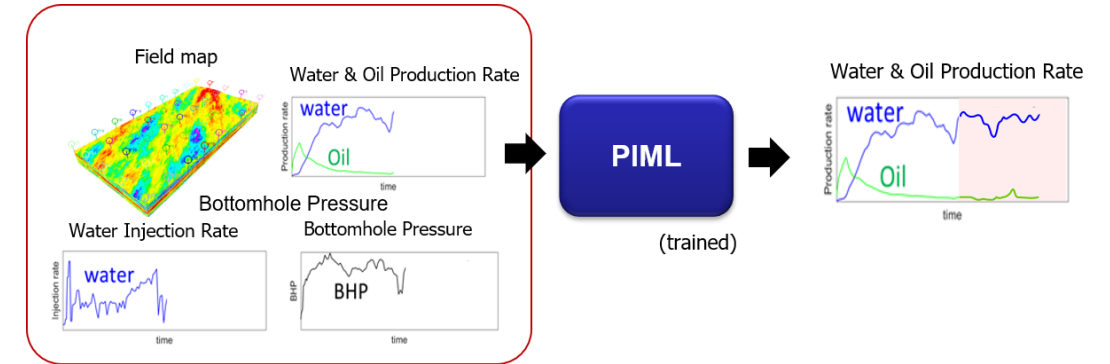


Fuel Burnup Nuclear Forensics using inverse-PINNs

Course projects demonstration (Cont'd)



Conditional Variational Auto-Encoder for Neural Thermal Scattering for On-the-Fly Evaluation of $S(\alpha, \beta)$



LSTM-PINN for Production Forecasting

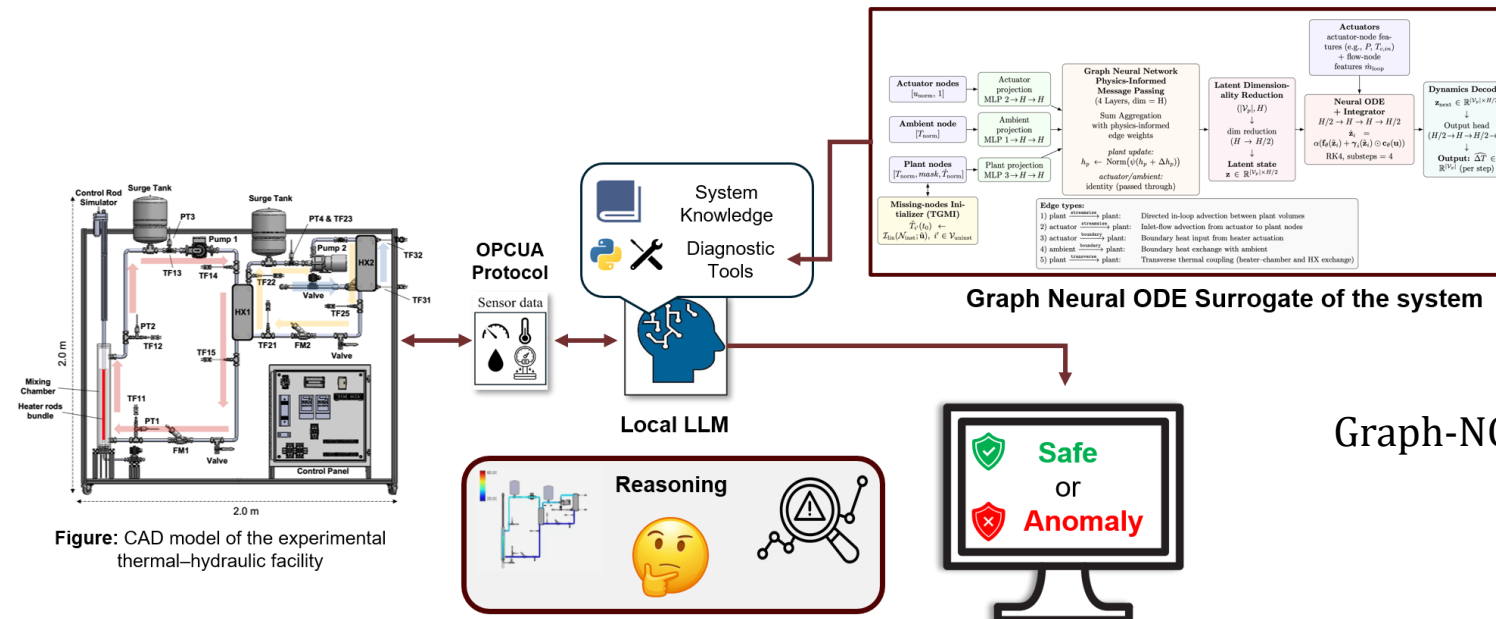
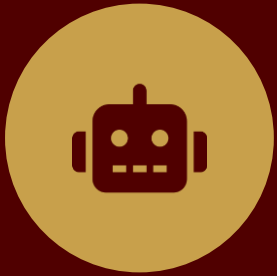


Figure: CAD model of the experimental thermal-hydraulic facility

Graph-NODE-backend DT for real-time prognostic of thermal-fluid facility



PART II · GENAI-ENHANCED TEACHING

Now: how we teach the course

Yang Liu serves as an Innovative Teaching Fellow on the AI track, funded by Texas A&M.

Using GenAI to lower the coding barrier

We use LLMs to annotate notebooks so students read code as concepts, not syntax.

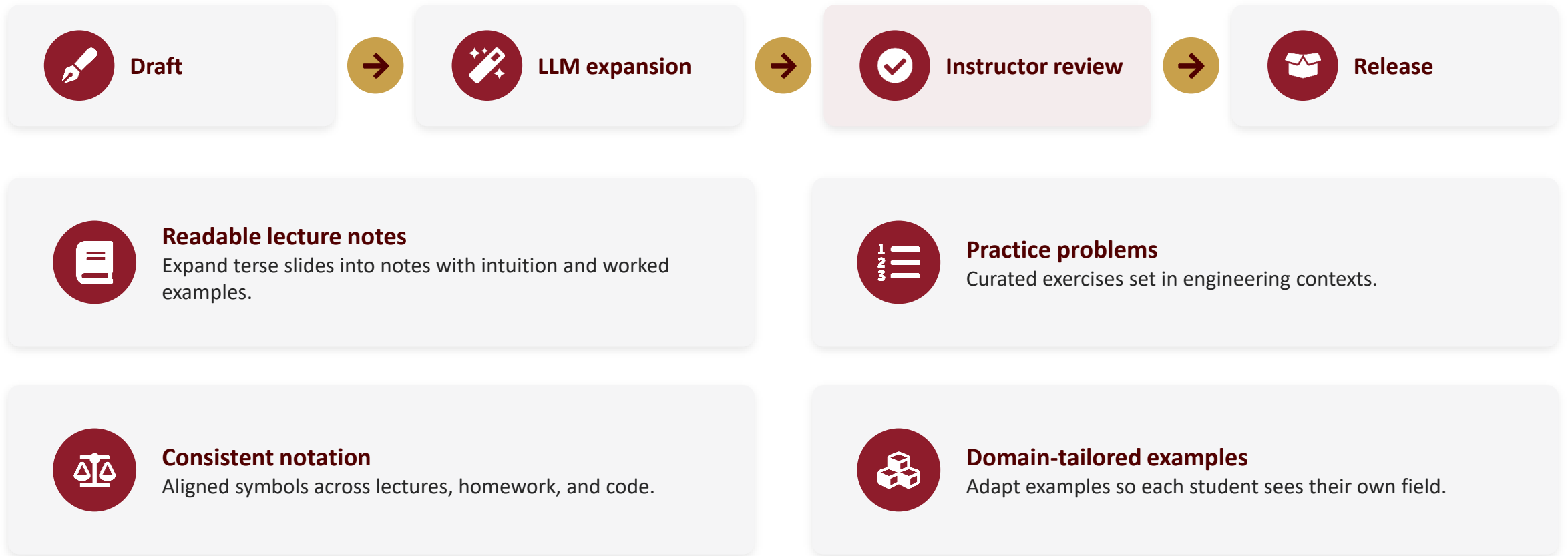
BEFORE · raw cell

```
opt = torch.optim.Adam(model.parameters(), lr=1e-3)
for ep in range(2000):
    opt.zero_grad()
    u = model(x)
    r = grad(u, x) - f(x)
    loss = (r**2).mean()
    loss.backward(); opt.step()
```

AFTER · GenAI-annotated

```
opt = torch.optim.Adam(model.parameters(), lr=1e-3)
for ep in range(2000):
    opt.zero_grad()      # clear old gradients
    u = model(x)         # network's estimate
    r = grad(u, x) - f(x) # PDE residual
    loss = (r**2).mean() # penalize error
    loss.backward()      # backpropagate
    opt.step()           # update weights
```

Using GenAI to strengthen course materials



The instructor stays in the loop: every generated item is reviewed before it reaches students.

Where we take GenAI-assisted teaching next



Course-specific AI teaching assistant

Retrieval over our notes, code, and assignments, with course-consistent answers.



Formative feedback on submissions

Rubric-aligned suggestions on student code and reports.



Personalized learning paths

Examples and difficulty that adapt to each student's background.



Agentic coding tutor

Guides students toward their own solution rather than solving for them.



Domain-tailored starter kits

Auto-generated project scaffolds matched to each field.

What makes engineering AI education work



Domain knowledge

The engineering problem comes first



Math foundations

Calculus and linear algebra



Deep learning methods

Scientific-computing tools that fit the problem

Thank you

[Email: y-liu@tamu.edu](mailto:y-liu@tamu.edu)

Research website: <https://smart.engr.tamu.edu/>

